

Artificial Potential Field and Extended Kalman Filter Methods

Equations and Source-Code Mapping

Contents

Abbreviations and full forms	3
1 Six-state vessel EKF	3
1.1 State vector and motion model	3
1.2 State-transition Jacobian	4
1.3 Prediction step	5
1.4 Pose and yaw-rate measurement models	5
1.5 Measurement update	6
2 Four-state obstacle constant-velocity EKF	6
2.1 State vector and constant-velocity prediction	6
2.2 Process noise	7
2.3 Position measurement update	7
2.4 Future position	8
3 APF obstacle geometry	8
3.1 Elliptical level set	8
3.2 Smooth elliptical repulsion	9
3.3 Observed dimensions in snapshot generation	10
4 APF attractive terms	10
4.1 Goal attraction	10
4.2 Path attraction	11
5 CPA-based dynamic-obstacle risk	12
6 COLREG encounter classification	13
7 Route-entry risk and virtual obstacle corridor	13

8	APF force composition and control output	14
8.1	Total force	14
8.2	Force direction and yaw-rate command	15
9	Processing chain	15
10	Implementation index	16

Abbreviations and full forms

Symbol or term	Meaning
N, E	Northing and easting coordinates.
G or ϕ	Vessel heading, in radians.
Δt	Discrete time step.
μ or $\mathbf{x}$	State estimate.
Σ or $\mathbf{P}$	State covariance matrix.
$\mathbf{Q}, \mathbf{R}$	Process-noise and measurement-noise covariance matrices.
$\mathbf{F}, \mathbf{H}$	Jacobians of the state-transition and measurement functions.
$\mathbf{z}$	Sensor measurement.
$\mathbf{K}$	Kalman gain.
Body frame	Vessel-fixed frame, with x forward and y to starboard.
Earth/NE frame	Earth-fixed north-east frame.
APF	Artificial Potential Field.
COLREGs	International Regulations for Preventing Collisions at Sea.
CPA	Closest point of approach.
DBSCAN	Density-Based Spatial Clustering of Applications with Noise.
TCPA, DCPA	Time to CPA and distance at CPA.
EKF	Extended Kalman Filter.
IMU	Inertial Measurement Unit.
LiDAR	Light Detection and Ranging.
PC	Principal Component.
PC1	First Principal Component.
PC2	Second Principal Component.
PCA	Principal Component Analysis.

Angular differences are normalised to $[-\pi, \pi)$ by `wrap_angle()`.

1 Six-state vessel EKF

In this chapter, N and E are northing and easting, G is the vessel heading, and dotted quantities are their time derivatives. Bold lower-case letters denote vectors and bold upper-case letters denote matrices. The subscripts b and e identify the body and earth frames, respectively; k is the discrete-time index. The symbols Δt , $\mathbf{u}$, $\mathbf{x}$, $\mathbf{p}$, $\mathbf{v}$, and $\mathbf{F}$ denote the sampling interval, thruster input, state, position, velocity, and force. The matrices $\mathbf{G}$, $\mathbf{R}_{be}(G)$, and $\mathbf{F}$ are, respectively, the thruster allocation matrix, the rotation from body to earth coordinates at heading G , and the state-transition Jacobian; the bold matrix $\mathbf{G}$ is distinguished from the scalar heading G by its typography. The functions f_{dyn} and f_{kin} are the vessel dynamics and rigid-body kinematics.

1.1 State vector and motion model

The vessel EKF uses the state vector

$$\mathbf{x} = \begin{bmatrix} N & E & G & \dot{N} & \dot{E} & \dot{G} \end{bmatrix}^T. \quad (1)$$

The thruster input is

$$\mathbf{u} = \begin{bmatrix} T_R & T_L \end{bmatrix}^T. \quad (2)$$

The allocation matrix maps the thruster forces to body-frame force and yaw moment,

$$\mathbf{F}_b = \mathbf{G}\mathbf{u}, \quad (3)$$

and the heading-dependent rotation maps this vector to the Earth-fixed frame,

$$\mathbf{F}_e = \mathbf{R}_{be}(G)\mathbf{F}_b. \quad (4)$$

The vessel dynamics produce the next Earth-frame velocity,

$$\mathbf{v}_{e,k+1} = f_{\text{dyn}}(\mathbf{v}_{e,k}, \mathbf{F}_e, \Delta t). \quad (5)$$

The Earth-frame velocity is transformed back to the body frame. Its surge and yaw-rate components form the kinematic input,

$$\mathbf{v} = v_b^x, \quad \omega = v_b^G. \quad (6)$$

The rigid-body kinematics then update the pose,

$$\mathbf{p}_{k+1} = f_{\text{kin}}(\mathbf{p}_k, [\mathbf{v}, \omega]^\top, \Delta t). \quad (7)$$

The implementation is located in `motion_model()`:

```
Fb = self.G @ control_input
H_eb = HomogeneousTransformation(state[N:E+1], state[G])
Fe = H_eb.H_R @ Fb

ve = self.robot.model(
    dynamics_translation_e,
    dynamics_rotation_e,
    Fe,
    state[DOTN:DOTG+1],
    dt,
)

vb = Inverse(H_eb.H_R) @ ve
u = Vector(2)
u[0, 0] = vb[0, 0]
u[1, 0] = vb[2, 0]

p = rigid_body_kinematics(state[N:G+1], u, dt)
p[2, 0] = p[2, 0] % (2 * np.pi)
```

1.2 State-transition Jacobian

The implementation uses a first-order discrete-time Jacobian. Here $\mathbf{F}_k$ is the Jacobian of the transition function at time k , and the unit entries preserve direct integration of northing, easting, and heading from their corresponding rates:

$$\mathbf{F} = \begin{bmatrix} 1 & 0 & 0 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 & \Delta t & 0 \\ 0 & 0 & 1 & 0 & 0 & \Delta t \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \quad (8)$$

```

F = Identity(6)
F[N, DOTN] = dt
F[E, DOTE] = dt
F[G, DOTG] = dt

```

1.3 Prediction step

The symbols $\hat{\mathbf{x}}_k$ and $\hat{\Sigma}_k$ denote the predicted state and covariance, while $\mathbf{x}_{k-1}$ and Σ_{k-1} are the previous corrected state and covariance. The transition function is f , $\mathbf{Q}_k$ is the process-noise covariance, and the superscript T denotes matrix transpose. The state and covariance predictions are

$$\hat{\mathbf{x}}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k, \Delta t), \quad (9)$$

$$\hat{\Sigma}_k = \mathbf{F}_k \Sigma_{k-1} \mathbf{F}_k^T + \mathbf{Q}_k. \quad (10)$$

```

def extended_kalman_filter_predict(mu, Sigma, u, f, Q, dt):
    pred_mu, F = f(mu, u, dt)
    pred_Sigma = F @ Sigma @ F.T + Q
    return pred_mu, pred_Sigma

```

1.4 Pose and yaw-rate measurement models

The measurement vector is $\mathbf{z}$, and h_{pose} is the pose measurement function. The matrix $\mathbf{H}$ is its measurement Jacobian. The scalar z_{gyro} is the measured yaw rate, and H_{gyro} is the corresponding Jacobian entry. The ArUco measurement contains northing, easting, and heading:

$$\mathbf{z}_{\text{pose}} = h_{\text{pose}}(\mathbf{x}) = \begin{bmatrix} N & E & G & 0 & 0 & 0 \end{bmatrix}^T. \quad (11)$$

Its Jacobian has unit entries at (N, N) , (E, E) , and (G, G) . The IMU model measures yaw rate:

$$z_{\text{gyro}} = \dot{G}, \quad H_{\text{gyro}}[\dot{G}, \dot{G}] = 1. \quad (12)$$

```

def h_pose_update(x):
    est_measurement = Vector(6)
    est_measurement[N] = x[N]
    est_measurement[E] = x[E]
    est_measurement[G] = x[G]
    H = Matrix(6, 6)
    H[N, N] = 1
    H[E, E] = 1
    H[G, G] = 1
    return est_measurement, H

def h_grate_update(x):
    est_measurement = Vector(6)
    est_measurement[DOTG] = x[DOTG]
    H = Matrix(6, 6)
    H[DOTG, DOTG] = 1
    return est_measurement, H

```

1.5 Measurement update

In this subsection, $\hat{z}$ is the predicted measurement, $\mathbf{y}$ is the innovation, $\mathbf{S}$ is the innovation covariance, and $\mathbf{K}$ is the Kalman gain. Superscripts $-$ and $+$ identify the predicted and corrected quantities. The matrix $\mathbf{I}$ is the identity matrix, and $\mathbf{R}$ is the measurement-noise covariance. The predicted measurement, innovation, innovation covariance, and Kalman gain are

$$\hat{z} = h(\hat{\mathbf{x}}), \quad (13)$$

$$\mathbf{y} = \mathbf{z} - \hat{z}, \quad (14)$$

$$\mathbf{S} = \mathbf{H}\Sigma^-\mathbf{H}^\top + \mathbf{R}, \quad (15)$$

$$\mathbf{K} = \Sigma^-\mathbf{H}^\top\mathbf{S}^{-1}. \quad (16)$$

The corrected state is

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\mathbf{y}. \quad (17)$$

The heading innovation is wrapped before correction:

$$y_G = \text{wrap}(z_G - \hat{z}_G). \quad (18)$$

The vessel EKF currently updates the covariance as

$$\Sigma^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\Sigma^-. \quad (19)$$

```
def extended_kalman_filter_update(
    mu, Sigma, z, h, R, wrap_index=None
):
    pred_z, H = h(mu)
    K = Sigma @ H.T @ Inverse(H @ Sigma @ H.T + R)
    delta_z = z - pred_z
    if wrap_index is not None:
        delta_z[wrap_index] = (
            delta_z[wrap_index] + np.pi
        ) % (2 * np.pi) - np.pi
    cor_mu = mu + K @ delta_z
    cor_Sigma = (Identity(mu.shape[0]) - K @ H) @ Sigma
    return cor_mu, cor_Sigma
```

In the main loop, the update order is thruster-based prediction, ArUco pose correction, and IMU yaw-rate correction.

2 Four-state obstacle constant-velocity EKF

The obstacle EKF contains no yaw, heading, or turn-rate state. At prediction index k , it estimates northing p_N , easting p_E , and their corresponding earth-frame velocities v_N and v_E . The symbol Δt denotes the prediction interval; $\mathbf{F}$, $\mathbf{Q}$, $\mathbf{H}$, $\mathbf{R}$, $\mathbf{P}$, and $\mathbf{K}$ denote the state-transition, process-noise, measurement, measurement-noise, covariance, and Kalman-gain matrices respectively.

2.1 State vector and constant-velocity prediction

Each tracked obstacle uses the four-state vector

$$\mathbf{x} = \begin{bmatrix} p_N & p_E & v_N & v_E \end{bmatrix}^\top. \quad (20)$$

Its constant-velocity transition is

$$\mathbf{x}_{k+1} = \mathbf{F}\mathbf{x}_k, \quad \mathbf{F} = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (21)$$

```
state = np.asarray(state, dtype=float).reshape(4)
covariance = np.asarray(covariance, dtype=float).reshape(4, 4)
F = np.array([
    [1.0, 0.0, dt, 0.0],
    [0.0, 1.0, 0.0, dt],
    [0.0, 0.0, 1.0, 0.0],
    [0.0, 0.0, 0.0, 1.0],
], dtype=float)
predicted_state = F @ state
predicted_covariance = F @ covariance @ F.T + self.obstacle_ekf_process_noise(dt)
```

The direction used to orient a predicted APF ellipse is derived from the estimated velocity vector when its magnitude is non-zero; it is not stored or filtered as an additional yaw state.

2.2 Process noise

The process covariance assumes white acceleration with standard deviation σ_a . With $q = \sigma_a^2$, the implemented covariance is

$$\mathbf{Q} = q \begin{bmatrix} \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} & 0 \\ 0 & \frac{\Delta t^4}{4} & 0 & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & 0 & \Delta t^2 & 0 \\ 0 & \frac{\Delta t^3}{2} & 0 & \Delta t^2 \end{bmatrix}. \quad (22)$$

```
q = self.obstacle_ekf_accel_std_m_s2 ** 2
dt2, dt3, dt4 = dt * dt, dt * dt * dt, (dt * dt) ** 2
return q * np.array([
    [0.25 * dt4, 0.0, 0.5 * dt3, 0.0],
    [0.0, 0.25 * dt4, 0.0, 0.5 * dt3],
    [0.5 * dt3, 0.0, dt2, 0.0],
    [0.0, 0.5 * dt3, 0.0, dt2],
], dtype=float)
```

2.3 Position measurement update

LiDAR supplies the two-dimensional obstacle position,

$$\mathbf{z} = \begin{bmatrix} p_N^{\text{meas}} & p_E^{\text{meas}} \end{bmatrix}^\top, \quad \mathbf{H} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}. \quad (23)$$

The correction uses the innovation $\boldsymbol{\nu} = \mathbf{z} - \mathbf{H}\mathbf{x}^-$ and the Joseph covariance form,

$$\mathbf{S} = \mathbf{H}\mathbf{P}^- \mathbf{H}^\top + \mathbf{R}, \quad \mathbf{K} = \mathbf{P}^- \mathbf{H}^\top \mathbf{S}^{-1}, \quad (24)$$

$$\mathbf{x}^+ = \mathbf{x}^- + \mathbf{K}\boldsymbol{\nu}, \quad \mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^- (\mathbf{I} - \mathbf{K}\mathbf{H})^\top + \mathbf{K}\mathbf{R}\mathbf{K}^\top. \quad (25)$$

```

H = np.array([[1.0, 0.0, 0.0, 0.0], [0.0, 1.0, 0.0, 0.0]])
innovation = measurement_ne - H @ state
innovation_covariance = H @ covariance @ H.T + R_obs
try:
    kalman_gain = covariance @ H.T @ np.linalg.inv(innovation_covariance)
except np.linalg.LinAlgError:
    kalman_gain = covariance @ H.T @ np.linalg.pinv(innovation_covariance)
corrected_state = state + kalman_gain @ innovation
correction = np.eye(4) - kalman_gain @ H
corrected_covariance = correction @ covariance @ correction.T + kalman_gain @ R_obs
    @ kalman_gain.T

```

The vessel's own yaw uncertainty can enlarge $\mathbf{R}$ because it rotates a LiDAR measurement into the earth frame. This is a measurement-uncertainty term only; it does not introduce yaw into the four-state obstacle EKF.

2.4 Future position

For a horizon $\tau \geq 0$, the code propagates a track with

$$\mathbf{p}(\tau) = \begin{bmatrix} p_N & p_E \end{bmatrix}^T + \tau \begin{bmatrix} v_N & v_E \end{bmatrix}^T. \quad (26)$$

```

state = np.asarray(track.get("state", [np.nan] * 4), dtype=float).reshape(4)
position_ne = state[:2]
velocity_ne = state[2:4]
return position_ne + velocity_ne * max(float(dt_s), 0.0), velocity_ne.copy()

```

3 APF obstacle geometry

In this chapter, $\mathbf{r}$ is the relative position from the obstacle centre to the vessel, while $\mathbf{l}$ and $\mathbf{w}$ are orthogonal unit vectors along the obstacle length and width. The observed principal-component dimensions $pc1$ and $pc2$ define the corresponding semi-axes a_0 and b_0 . The scalars a and b are the projections of $\mathbf{r}$ onto these axes, ρ is the dimensionless elliptical level, $\mathbf{g}$ is the outward gradient, and $\hat{\mathbf{d}}$ is its unit direction. The scalar s is the outer field scale, u is the normalised decay coordinate, f_{fade} is the smooth fade factor, k_{rep} is the repulsive gain, and $\mathbf{F}_{\text{rep}}$ is the resulting repulsive force. A hat denotes a unit vector, $\|\cdot\|$ denotes Euclidean norm, and $\cdot$ denotes the inner product.

3.1 Elliptical level set

The measured obstacle dimensions $pc1$ and $pc2$ are obtained from the first and second principal components of the clustered LiDAR points. The axial projections are

$$a = \mathbf{r} \cdot \mathbf{l}, \quad b = \mathbf{r} \cdot \mathbf{w}. \quad (27)$$

The APF semi-axes are defined from the observed dimensions and enlarged by the own-vessel equivalent radius r_{own} :

$$a_0 = \frac{pc1}{2} + r_{\text{own}}, \quad b_0 = \frac{pc2}{2} + r_{\text{own}}. \quad (28)$$

Minimum PC dimensions are used only when an observation is invalid or unavailable. The elliptical level is

$$\rho = \sqrt{\left(\frac{a}{a_0}\right)^2 + \left(\frac{b}{b_0}\right)^2}. \quad (29)$$

The outward gradient and corresponding unit direction are

$$\mathbf{g} = \frac{a}{a_0^2} \mathbf{l} + \frac{b}{b_0^2} \mathbf{w}, \quad (30)$$

$$\hat{\mathbf{d}} = \frac{\mathbf{g}}{\|\mathbf{g}\|}. \quad (31)$$

The condition $\rho < 1$ indicates that the vessel has entered the observed obstacle boundary. The repulsive field is inactive when $\rho \geq s$.

```
pc1_m, pc2_m = self.obstacle_pc_dimensions(obstacle)
semi_length_m = 0.5 * pc1_m + self.apf_own_equivalent_radius_m
semi_width_m = 0.5 * pc2_m + self.apf_own_equivalent_radius_m

along = float(np.dot(offset, length_axis))
across = float(np.dot(offset, width_axis))
level = float(np.hypot(
    along / semi_length_m,
    across / semi_width_m,
))
gradient = (
    along / (semi_length_m * semi_length_m) * length_axis
    + across / (semi_width_m * semi_width_m) * width_axis
)
away = gradient / np.linalg.norm(gradient)
```

3.2 Smooth elliptical repulsion

For $s > 1$, the implementation applies a cubic smoothstep decay. The variable u clips the level interval $[1, s]$ to $[0, 1]$, so that the fade remains bounded both inside the obstacle and outside the influence region:

$$u = \text{clip}\left(\frac{\rho - 1}{s - 1}, 0, 1\right), \quad (32)$$

$$f_{\text{fade}} = 1 - (3u^2 - 2u^3), \quad (33)$$

$$\mathbf{F}_{\text{rep}} = k_{\text{rep}} f_{\text{fade}} \hat{\mathbf{d}}. \quad (34)$$

The force is maximal at and inside $\rho = 1$, and it decreases smoothly to zero at $\rho = s$. To retain the measured size information in the force magnitude, the implementation scales the gain by the square root of the PCA footprint, while clipping the multiplier to a bounded range:

$$k_{\text{eff}} = k_{\text{rep}} \text{clip}\left(\sqrt{pc1 pc2}, 0.5, 2.0\right). \quad (35)$$

```
u = np.clip(
    (level - 1.0) / (outer_scale - 1.0),
    0.0,
    1.0,
)
fade = 1.0 - (3.0 * u * u - 2.0 * u * u * u)
return gain * fade * np.asarray(away).reshape(2), True
```

The corresponding controller code evaluates the ellipse at the actual measured or virtual-field position. Hence DCPA remains a diagnostic quantity; it is not treated as a second force location.

```

level, away = ellipse_level_and_away(
    -centre_body, axis_body,
    0.5 * pc1_m + self.apf_own_equivalent_radius_m,
    0.5 * pc2_m + self.apf_own_equivalent_radius_m,
)
if level >= outer_scale:
    return np.zeros(2, dtype=float), False
size_gain = np.sqrt(max(pc1_m * pc2_m, 1e-6))
gain = self.apf_repulsive_gain * np.clip(size_gain, 0.5, 2.0)
return smooth_ellipse_repulsion(level, away, gain, outer_scale)

```

The project uses two fields with this form. The first is centred on the current LiDAR obstacle, while the second is centred on the EKF/CPA-predicted obstacle. Their outer scales are selected from encounter-specific profiles: head-on, overtaking, crossing, and static-obstacle cases. Predicted fields use a separate profile table from current-observation fields.

3.3 Observed dimensions in snapshot generation

The offline potential-field renderer uses the same observed $pc1$ and $pc2$ dimensions. The stored equivalent radius is not used to reconstruct the obstacle dimensions when these observations are present; fallback values are the configured minimum PC dimensions.

```

pc1_m = positive(obstacle.get("pc1_m"), minimum_pc1_m)
pc2_m = positive(obstacle.get("pc2_m"), minimum_pc2_m)
return max(pc1_m, minimum_pc1_m), max(
    min(pc2_m, pc1_m), minimum_pc2_m
)

```

4 APF attractive terms

In this chapter, all attractive-force vectors are expressed in the vessel body frame. The target vector is $\mathbf{t}$, its distance is d , k_g is the goal gain, and d_{sat} is the maximum distance used for saturating the goal-force magnitude. The path variables $\mathbf{s}$, $\mathbf{g}$, $\mathbf{p}$, d_{path} , L_{path} , λ , and $\mathbf{p}_{\text{path}}$ denote the start, goal, current position, route vector, route length, clipped projection coefficient, and selected route point. The transformation $T_{\text{earth} \rightarrow \text{body}}$ maps earth-frame vectors into body-frame vectors. The path error and its norm are $\mathbf{e}_{\text{path}}$ and d_{path} ; k_p is the path gain, and $d_{\text{threshold}}$ is the path dead-band threshold. Finally, $\mathbf{F}_{\text{goal}}$, $\mathbf{F}_{\text{path}}$, and $\mathbf{F}_{\text{att}}$ denote the goal, path, and total attractive forces.

4.1 Goal attraction

Let $\mathbf{t}$ be the target position in the body frame, with $d = \|\mathbf{t}\|$. The saturated goal-attraction term is

$$\mathbf{F}_{\text{goal}} = k_g \min(d, d_{\text{sat}}) \frac{\mathbf{t}}{d}. \quad (36)$$

```

distance_m = float(np.linalg.norm(target_body))
magnitude = self.apf_goal_gain * min(
    distance_m,

```

```

        self.apf_attraction_saturation_m,
    )
    goal_force = magnitude * target_body / distance_m

```

4.2 Path attraction

The path-attraction field keeps the vessel close to the straight route from the initial position to the goal. Let $\mathbf{s}$ and $\mathbf{g}$ denote the start and goal positions in the north-east frame, and let $\mathbf{p}$ be the current vessel position. The route vector and its squared length are

$$\begin{aligned}\mathbf{d}_{\text{path}} &= \mathbf{g} - \mathbf{s}, \\ L_{\text{path}}^2 &= \mathbf{d}_{\text{path}}^\top \mathbf{d}_{\text{path}}.\end{aligned}\tag{37}$$

If L_{path}^2 is smaller than a numerical tolerance, the start and goal are treated as coincident and the path-attraction force is set to zero.

For a valid route, the projection coefficient of the current position onto the start-to-goal segment is

$$\lambda^* = \frac{(\mathbf{p} - \mathbf{s})^\top \mathbf{d}_{\text{path}}}{L_{\text{path}}^2}.\tag{38}$$

The implementation clips this coefficient to $[0, 1]$:

$$\lambda = \text{clip}(\lambda^*, 0, 1).\tag{39}$$

Consequently, the reference point is on the finite route segment rather than on its infinite extension,

$$\mathbf{p}_{\text{path}} = \mathbf{s} + \lambda \mathbf{d}_{\text{path}}.\tag{40}$$

When the perpendicular projection falls before the start or beyond the goal, the start or goal endpoint is therefore used as the reference point.

The vector from the vessel to this reference point is first calculated in the north-east frame and then transformed into the vessel body frame. With $T_{\text{earth} \rightarrow \text{body}}$ denoting the rotation from the earth frame to the body frame,

$$\mathbf{e}_{\text{path}} = T_{\text{earth} \rightarrow \text{body}}(\mathbf{p}_{\text{path}} - \mathbf{p}).\tag{41}$$

The norm $d_{\text{path}} = \|\mathbf{e}_{\text{path}}\|$ is the current cross-track distance to the selected route point. The resulting path force is

$$\mathbf{F}_{\text{path}} = \begin{cases} \mathbf{0}, & d_{\text{path}} < d_{\text{threshold}}, \\ k_p \mathbf{e}_{\text{path}}, & d_{\text{path}} \geq d_{\text{threshold}}. \end{cases}\tag{42}$$

The dead-band threshold prevents small position-estimation fluctuations from causing continuous corrective commands while the vessel is already close to the route. Outside this band, the force magnitude is proportional to the cross-track error and its direction points toward the route.

```

path_vec = self.goal_ne - self.start_ne
path_len_sq = float(np.dot(path_vec, path_vec))
if path_len_sq < 1e-9:
    return np.zeros(2, dtype=float)

current_ne = np.array(
    [float(self.North), float(self.East)], dtype=float
)
ratio = float(np.clip(

```

```

    np.dot(current_ne - self.start_ne, path_vec)
    / path_len_sq,
    0.0,
    1.0,
))
closest_ne = self.start_ne + ratio * path_vec
to_path_body = self.earth_point_to_body(closest_ne)
distance_m = float(np.linalg.norm(to_path_body))

if distance_m < self.apf_path_threshold_m or distance_m < 1e-6:
    return np.zeros(2, dtype=float)
return self.apf_path_gain * to_path_body

```

The total attractive term is the vector sum of the two components:

$$\mathbf{F}_{\text{att}} = \mathbf{F}_{\text{goal}} + \mathbf{F}_{\text{path}}. \quad (43)$$

5 CPA-based dynamic-obstacle risk

In this chapter, $\mathbf{p}$ is the obstacle position relative to the vessel, $\mathbf{v}_{\text{obs}}$ and $\mathbf{v}_{\text{own}}$ are the obstacle and own- vessel velocities, and $\mathbf{v}_{\text{rel}}$ is their relative velocity. TCPA is the non-negative time to the closest point of approach, and DCPA is the distance at that time. The symbols $\mathbf{p}(t)$ and $\mathbf{v}_{\text{rel}}$ describe relative position and velocity, while $\|\cdot\|$ denotes vector magnitude. The zero-speed threshold is a numerical guard used when the relative-velocity norm is too small.

The relative velocity is

$$\mathbf{v}_{\text{rel}} = \mathbf{v}_{\text{obs}} - \mathbf{v}_{\text{own}}. \quad (44)$$

If $\|\mathbf{v}_{\text{rel}}\|^2$ is sufficiently small, the implementation returns

$$\text{TCPA} = \infty, \quad \text{DCPA} = \|\mathbf{p}\|. \quad (45)$$

Otherwise,

$$\text{TCPA} = \max\left(-\frac{\mathbf{p} \cdot \mathbf{v}_{\text{rel}}}{\|\mathbf{v}_{\text{rel}}\|^2}, 0\right), \quad (46)$$

$$\text{DCPA} = \|\mathbf{p} + \mathbf{v}_{\text{rel}} \text{TCPA}\|. \quad (47)$$

```

rel_vel = obs_vel_body - own_vel_body
rel_speed_sq = float(np.dot(rel_vel, rel_vel))

if rel_speed_sq < 1e-9:
    return np.inf, float(np.linalg.norm(obs_pos_body))

tcpa = max(
    -float(np.dot(obs_pos_body, rel_vel)) / rel_speed_sq,
    0.0,
)
dcpa = float(np.linalg.norm(
    obs_pos_body + rel_vel * tcpa
))
return tcpa, dcpa

```

6 COLREG encounter classification

The encounter classifier first uses the obstacle bearing in the vessel frame, then tests the obstacle course and the speed relation. The sectors are “ahead” ($[-67.5^\circ, 67.5^\circ]$), port, starboard, and astern. A head-on encounter is accepted only when an ahead obstacle has a reciprocal course. Overtaking is deliberately stricter: it requires a narrow ahead sector, an approximately common course, a verified stern-side geometry, and an own-vessel speed exceeding the obstacle speed. This avoids temporarily misclassifying a crossing vessel as an overtaking vessel when a track jitters near a sector boundary.

```
if head_on_ahead and head_on_reciprocal:
    return "head_on", -1.0, "COLREG Rule 14: both alter to starboard"

if ahead and (
    speed_known and overtaking_ahead and overtaking_same_course
    and own_in_obstacle_stern_sector
    and own_speed_m_s > obstacle_speed_m_s + 0.08
    and obstacle_lateral_speed_m_s <= 0.06
):
    return "overtaking", -1.0, "COLREG Rule 13: overtake on starboard side"

if on_starboard:
    return ("crossing_from_starboard", -1.0,
           "COLREG Rule 15: give way, pass astern")
```

For Rule 13, the controller uses a short-horizon EKF prediction only when the track is stable and its heading uncertainty is below the configured bound. Rule 14 may use the measured direction before this confirmation so that an approaching vessel is not missed during track initialisation.

7 Route-entry risk and virtual obstacle corridor

The controller does not create a single virtual field at DCPA. For every stable dynamic EKF track with $0 < \text{TCPA} \leq T_{\text{horizon}}$, it places the final virtual field at 2 TCPA and fills the interval from the current track position to that field with overlapping ellipses. The bridge turns the predicted motion into a continuous exclusion corridor, allowing route planning to respond before reaching the closest-approach point. The final field centre is

$$\mathbf{p}_{\text{virtual}} = \mathbf{p}_{\text{obs}} + 2 \text{ TCPA } \mathbf{v}_{\text{obs}}. \quad (48)$$

The route-entry check maps the line segment into the ellipse coordinate system. With $\mathbf{q}_0$ and $\mathbf{q}_1$ denoting the normalised start offset and direction, the route is unsafe when

$$\min_{0 \leq \lambda \leq 1} \|\mathbf{q}_0 + \lambda \mathbf{q}_1\| \leq 1. \quad (49)$$

```
final_ne = start_ne + velocity_ne * (2.0 * tcpa_s)
bridge_count = max(int(np.ceil(
    np.linalg.norm(final_ne - start_ne) / max_gap_m
)) - 1, 0)
for index in range(1, bridge_count + 1):
    self.apf_virtual_obstacles.append(field(
        start_ne + (final_ne - start_ne) * index / (bridge_count + 1),
        bridge=True,
    ))
```

```
self.apf_virtual_obstacles.append(field(final_ne))
```

```
axis /= axis_norm
lateral = np.array([-axis[1], axis[0]])
scale = np.array([half_long_m + margin_m, half_lateral_m + margin_m])
offset = np.array([np.dot(start-centre, axis), np.dot(start-centre, lateral)]) /
    scale
direction = np.array([np.dot(end-start, axis), np.dot(end-start, lateral)]) / scale
denominator = float(np.dot(direction, direction))
alpha = 0.0 if denominator < 1e-12 else float(np.clip(
    -np.dot(offset, direction) / denominator, 0.0, 1.0
))
return np.linalg.norm(offset + alpha * direction) <= 1.0
```

8 APF force composition and control output

In this chapter, $\mathbf{F}$ is the total body-frame guidance force, $\mathbf{F}_{\text{att}}$ is the attractive force, and $\mathbf{F}_{\text{rep}}$ is the accumulated repulsive force. When a COLREG side is locked, $\mathbf{F}_{\text{offset}}$ pulls the temporary target to that side of the main route. In the steering equations, F_x and F_y are the forward and lateral components of $\mathbf{F}$, θ_F is its steering angle, k_h is the heading gain, ω_c is the commanded yaw rate, and ω_{max} is its magnitude limit. The operator `atan2` returns the signed angle from the two force components, while `clip` limits a scalar to a prescribed interval. The final command contains surge speed v_c and yaw rate ω_c .

8.1 Total force

The controller sums the attractive force and the repulsive contributions:

$$\mathbf{F} = \mathbf{F}_{\text{att}} + \sum_i \mathbf{F}_{\text{rep},i}. \quad (50)$$

When a COLREG side is locked, the route look-ahead target is shifted by the required clearance in the selected route normal direction and contributes an additional attractive term,

$$\mathbf{F} \leftarrow \mathbf{F} + \mathbf{F}_{\text{offset}}. \quad (51)$$

If an active field would make the forward component non-positive, the steering vector is repaired with a small positive forward component and a lateral sign consistent with the locked avoidance side. This preserves turning authority without reversing the vessel.

```
force_body = attractive_force + repulsive_force
if any_repulsion and self.apf_side_lock_sign != 0.0:
    offset_target_ne = target_ne + (
        self.apf_side_lock_sign * clearance_offset_m * route_normal_left_ne
    )
    offset_target_body = self.earth_point_to_body(offset_target_ne)
    offset_force = self.apf_clearance_gain * (
        offset_target_body / max(np.linalg.norm(offset_target_body), 1e-6)
    )
    force_body += offset_force

if any_repulsion and force_body[0] <= 0.0:
    steering_force[1] = side_sign * max(abs(force_body[1]), 0.20)
    steering_force[0] = max(0.25 * abs(steering_force[1]), 0.05)
```

The controller also preserves the COLREGs-selected avoidance side. If the forward force becomes non-positive, it retains a lateral component and inserts a small positive forward component.

8.2 Force direction and yaw-rate command

The steering angle is

$$\theta_F = \text{atan2}(F_y, F_x). \quad (52)$$

After limiting this angle, the yaw-rate command is

$$\omega_c = \text{clip}\left(-\frac{k_h \theta_F}{\Delta t}, -\omega_{\max}, \omega_{\max}\right). \quad (53)$$

```
force_angle = np.arctan2(
    steering_force[1], steering_force[0]
)
force_angle = np.clip(
    force_angle,
    -self.apf_heading_step_limit_rad,
    self.apf_heading_step_limit_rad,
)
u_cmd[1, 0] = np.clip(
    -self.apf_heading_gain * force_angle
    / max(self.lasdt, 1e-3),
    -self.w_max,
    self.w_max,
)
```

During APF avoidance, the force magnitude does not directly determine surge speed. The active encounter profile supplies a conservative descent speed, and a close overtaking manoeuvre can further reduce it while retaining the minimum turning angle:

```
surge_speed = float(active_params.get(
    "constant_descent_speed_m_s",
    self.apf_constant_descent_speed_m_s,
))
force_angle, surge_speed = self.apf_overtaking_close_turn_command(
    primary_encounter,
    nearest_obstacle_distance_m,
    force_angle,
    surge_speed,
)
u_cmd[0, 0] = float(np.clip(surge_speed, 0.0, self.v_max))
```

The final APF control output is

$$\mathbf{u}_{\text{APF}} = \begin{bmatrix} v_c & \omega_c \end{bmatrix}^T. \quad (54)$$

9 Processing chain

The complete processing chain is:

1. cluster the LiDAR point cloud using DBSCAN and estimate obstacle dimensions using PCA;

2. associate detections with obstacle tracks;
3. predict and correct each obstacle state with the four-state constant-velocity EKF;
4. compute obstacle velocity, TCPA, DCPA, and future positions;
5. classify the encounter using the COLREG bearing, course, and speed tests;
6. construct the current field and the EKF $2 \times \text{TCPA}$ virtual-field corridor;
7. reject route segments entering an APF ellipse and combine APF attraction, repulsion, and the locked COLREG side constraint;
8. convert the resultant-force direction into surge and yaw-rate commands; and
9. allocate the commanded force and moment to the two thrusters.

The vessel EKF supplies the vessel position, heading, and velocity at the start of this chain. The obstacle EKF supplies the predicted dynamic-obstacle state used by the APF.

10 Implementation index

Function	Source-code entry point
Vessel motion model	<code>motion_model</code>
Vessel EKF prediction	<code>extended_kalman_filter_predict</code>
Vessel EKF update	<code>extended_kalman_filter_update</code>
Pose measurement model	<code>h_pose_update</code>
Yaw-rate measurement model	<code>h_grate_update</code>
Obstacle CV propagation	<code>obstacle_ekf_predict</code>
Obstacle EKF process noise	<code>obstacle_ekf_process_noise</code>
Obstacle EKF measurement update	<code>obstacle_ekf_update</code>
Elliptical geometry	<code>ellipse_level_and_away</code>
Smooth elliptical repulsion	<code>smooth_ellipse_repulsion</code>
Goal and path attraction	<code>apf_goal_attraction_body,</code> <code>apf_path_attraction_body</code>
CPA computation	<code>apf_cpa_metrics</code>
COLREG zone classification	<code>classify_colreg_zone,</code> <code>apf_classify_encounter</code>
Virtual-field corridor	<code>update_apf_virtual_obstacles</code>
Route-ellipse intersection	<code>segment_intersects_ellipse</code>
Obstacle repulsion composition	<code>apf_repulsion_for_obstacle</code>
APF control output	<code>compute_apf_control</code>